

Python Programming

(Course Code: UE24CS1204)

Lab Manual

For II Semester

VISION & MISSION OF THE GMU

VISION

GM University will have a transformative impact on society through continual innovation in education, research, skill development, creativity, and entrepreneurship.

MISSION

- To disseminate knowledge and conduct research in academic areas such as science, engineering, commerce, management, health, humanities & social sciences, and legal studies with learner centric approach.
- To teach skills such as critical thinking, creativity and innovation, collaboration, communication, technical and digital, flexibility and adaptability, cultural values, and leadership and responsibility
- To develop global citizens by educating students on emotional, physical, social, economic, environmental, spiritual dimensions of human growth in addition to intellectual pursuits
- To address real-world challenges and to establish the groundwork for entrepreneurship and lifelong learning.



Python Laboratory Problem Statements

1. Python Basics & String Manipulation

Problem: Develop a Python program that takes a user's full name as input and performs the following operations:

- Extracts and prints the first and last name separately using **indexing and slicing**.
- Concatenates them in reverse order (Last Name, First Name).
- Converts the full name to uppercase and lowercase.
- Formats and prints a welcome message using **f-strings** and `.format()`.

2. Numeric Operations & Operator Precedence

Problem: Write a Python program that takes three numerical inputs (integer, float, and string representing a number), performs the following operations, and displays the results:

- Converts the string input to a float and an integer.
- Computes the sum, difference, product, and division of all three numbers.
- Demonstrates operator precedence by implementing expressions with `+`, `-`, `*`, `/`, `//`, `%`, and `**`.

3. Control Structures & Decision Making

Problem: Create a Python program that determines whether a given year is a **leap year** or not.

- Accept user input for a year.
- Use **if-elif-else** conditions to check leap year rules.
- Display an appropriate message based on the condition evaluation.

4. Looping Constructs & Range Iteration

Problem: Develop a Python program that generates and prints the **Fibonacci sequence** up to n terms, where n is provided by the user.

- Implement a **for loop** to compute and store the sequence in a list.
 - Use a **while loop** to print each term dynamically.
 - Allow the user to exit early by entering a negative number.
-



5. List & Tuple Operations

Problem: Create a Python program that manages a **student database** using lists and tuples.

- Use a list to store multiple student records, where each student is represented as a tuple containing (StudentID, Name, Marks).
 - Implement functions to:
 - **Add a student record**
 - **Search for a student by ID**
 - **Calculate and display the average marks**
 - **Remove a student record**
-

6. Dictionary & Set Operations

Problem: Write a Python program to perform operations on an **employee database** using dictionaries and sets.

- Store employee details as key-value pairs (EmployeeID: Name).
 - Implement functionalities to:
 - **Add and remove employees**
 - **Search for an employee by ID**
 - **Display all employee names in sorted order**
 - Use sets to manage employee **departments**, performing union, intersection, and difference operations on two departments.
-

7. Arrays & NumPy Operations

Problem: Implement a Python program that demonstrates **array operations** using NumPy.

- Create a NumPy array from user input.
 - Perform **element-wise addition, subtraction, multiplication, and division** on two arrays.
 - Compute **sum, mean, maximum, and minimum** values of the array.
 - Use slicing to extract specific sections of the array.
-

8. Pandas for Tabular Data Processing

Problem: Develop a Python program that reads and processes a CSV file containing **student marks** using Pandas.



- Read the dataset into a Pandas **DataFrame**.
 - Display the **top 5 and bottom 5 rows** of the dataset.
 - Filter and display students who scored above 80 marks.
 - Compute the **average, highest, and lowest** marks per subject.
 - Write the processed data into a new CSV file.
-

9. Data Visualization with Matplotlib

Problem: Write a Python program that visualizes **sales data** using Matplotlib.

- Accept sales data (product names and sales figures) from the user.
 - Generate the following visualizations:
 - **Bar chart** for sales comparison.
 - **Pie chart** showing percentage contribution of each product.
 - **Line plot** depicting sales trends over time.
 - Customize plots with titles, labels, legends, and colors.
 - Export the graphs as images.
-

10. Combined Project: Exploratory Data Analysis (EDA) on a Real Dataset

Problem:

Perform **Exploratory Data Analysis (EDA)** on a dataset (e.g., population, COVID-19 statistics, stock market data) using Pandas, NumPy, and Matplotlib.

- **Load** and display key statistics of the dataset.
- **Clean and preprocess** data by handling missing values.
- **Visualize trends and distributions** using scatter plots, histograms, and bar charts.
- **Derive insights and document findings.**



Lab Program 1: String Manipulation in Python

Objective:

- To understand **string indexing, slicing, concatenation, case conversion, and string formatting** in Python.

Problem Statement:

Develop a Python program that takes a user's full name as input and performs the following operations:

- Extracts and prints the **first and last name separately** using indexing and slicing.
- Concatenates them in **reverse order** (Last Name, First Name).
- Converts the **full name to uppercase and lowercase**.
- Formats and prints a **welcome message** using f-strings and .format().

Algorithm:

1. Take the user's full name as input.
2. Extract the first and last name using split() or slicing.
3. Print the first and last name separately.
4. Concatenate the names in reverse order (Last Name, First Name) and print.
5. Convert the full name to uppercase and lowercase, then print both versions.
6. Display a formatted welcome message using f-strings and .format().

Python Program:

```
1  ## Lab Program 1: String Manipulation in Python
2  # Taking user input
3  full_name = input("Enter your full name: ")
4
5  # Validating input: Ensure at least two words are entered
6  while " " not in full_name:    # or len(full_name.split()) < 2:
7      print("Error: Please enter at least your first and last name.")
8      full_name = input("Enter your full name: ")
9
10 # Splitting first and last name
11 name_parts = full_name.split()
12 first_name = name_parts[0]    # First word as first name
13 last_name = name_parts[-1]    # Last word as last name
```



```
15 # Printing extracted names
16 print("-----\nPrinting extracted names")
17 print("First Name:", first_name)
18 print("Last Name:", last_name)
19
20 # Reversing order
21 print("-----\nReversed Order (Last, First):", last_name, first_name)
22
23 # Converting to uppercase and lowercase
24 print("-----\nUppercase:", full_name.upper())
25 print("Lowercase:", full_name.lower())
26
27 # Using string formatting
28 print("-----\nWelcome, {}".format(full_name))
29 print(f"Welcome, {full_name}")
```

Sample Output:

```
Enter your full name: Raaja Raam Raheem
-----
Printing extracted names
First Name: Raaja
Last Name: Raheem
-----
Reversed Order (Last, First): Raheem Raaja
-----
Uppercase: RAAJA RAAM RAHEEM
Lowercase: raaja raam raheem
-----
Welcome, Raaja Raam Raheem
Welcome, Raaja Raam Raheem
```

```
Enter your full name: Raam
Error: Please enter at least your first and last name.
Enter your full name: Raaja R
-----
Printing extracted names
First Name: Raaja
Last Name: R
-----
Reversed Order (Last, First): R Raaja
-----
Uppercase: RAAJA R
Lowercase: raaja r
-----
Welcome, Raaja R
Welcome, Raaja R
```

```
Enter your full name: Raaja Raam
-----
Printing extracted names
First Name: Raaja
Last Name: Raam
-----
Reversed Order (Last, First): Raam Raaja
-----
Uppercase: RAAJA RAAM
Lowercase: raaja raam
-----
Welcome, Raaja Raam
Welcome, Raaja Raam
```



Viva Questions & Answers

1. What is string slicing in Python?

String slicing extracts a portion of a string using the syntax `string[start:end:step]`.

2. How do you extract a specific character from a string?

Use indexing, e.g., `string[index]` to get a character at a specific position.

3. What is the difference between `.format()` and f-strings?

`.format()` is method-based string formatting, while f-strings (`f"{var}"`) are more concise and efficient.

4. How do you convert a string to uppercase in Python?

Use `string.upper()` to convert all characters to uppercase.

5. What does `split()` do in Python?

It splits a string into a list of words based on a given delimiter (default is whitespace).

Lab Program 2: Numeric Operations

Objective:

To develop a Python program that performs basic numeric operations (addition, subtraction, multiplication, and division) on two user-input values with proper validation.

Problem Statement:

Write a Python program that takes three numerical inputs (integer, float, and string representing a number), performs the following operations, and displays the results:

- Converts the string input to a float and an integer.
- Computes the sum, difference, product, and division of all three numbers.

Algorithm:

1. Start the program.
2. Prompt the user to enter two numeric values.
3. Validate that both inputs are numbers.
4. Perform the following operations:
 - Addition



- Subtraction
 - Multiplication
 - Division (Ensure the denominator is not zero)
5. Display the results of each operation.
 6. End the program.

Python Program:

```
1  ## Lab Program 2: Numeric Operations
2  # Numeric Operations with Validation
3
4  # Prompt user for first number
5  num1 = input("Enter the first number: ")
6
7  # Validate if input is numeric
8  while not num1.replace('.', '', 1).isdigit():
9      print("Invalid input, Please enter a valid number.")
10     num1 = input("Enter the first number: ")
11  num1 = float(num1)
12
13 # Prompt user for second number
14 num2 = input("Enter the second number: ")
15
16 # Validate if input is numeric
17 while not num2.replace('.', '', 1).isdigit():
18     print("Invalid input! Please enter a valid number.")
19     num2 = input("Enter the second number: ")
20 num2 = float(num2)
21
22 # Performing numeric operations
23 addition = num1 + num2
24 subtraction = num1 - num2
25 multiplication = num1 * num2
26
27 # Handling division (prevent division by zero)
28 if num2 == 0:
29     division = "Undefined (Cannot divide by zero)"
30 else:
31     division = num1 / num2
32
```




```
33 # Display results
34 print("\nResults of Numeric Operations:")
35 print(f"Addition: {num1} + {num2} = {addition}")
36 print(f"Subtraction: {num1} - {num2} = {subtraction}")
37 print(f"Multiplication: {num1} * {num2} = {multiplication}")
38 print(f"Division: {num1} / {num2} = {division}")
```

Sample Output:

```
Enter the first number: 12
Enter the second number: 2
```

```
Results of Numeric Operations:
Addition: 12.0 + 2.0 = 14.0
Subtraction: 12.0 - 2.0 = 10.0
Multiplication: 12.0 * 2.0 = 24.0
Division: 12.0 / 2.0 = 6.0
```

```
Enter the first number: 3.2
Enter the second number: 1.2
```

```
Results of Numeric Operations:
Addition: 3.2 + 1.2 = 4.4
Subtraction: 3.2 - 1.2 = 2.0
Multiplication: 3.2 * 1.2 = 3.84
Division: 3.2 / 1.2 = 2.6666666666666667
```

```
Enter the first number: 12
Enter the second number: 0
```

```
Results of Numeric Operations:
Addition: 12.0 + 0.0 = 12.0
Subtraction: 12.0 - 0.0 = 12.0
Multiplication: 12.0 * 0.0 = 0.0
Division: 12.0 / 0.0 = Undefined (Cannot divide by zero)
```

```
Enter the first number: 12.3.4
Invalid input, Please enter a valid number.
Enter the first number: 12..3
Invalid input, Please enter a valid number.
Enter the first number: 12
Enter the second number: 2
```

```
Results of Numeric Operations:
Addition: 12.0 + 2.0 = 14.0
Subtraction: 12.0 - 2.0 = 10.0
Multiplication: 12.0 * 2.0 = 24.0
Division: 12.0 / 2.0 = 6.0
```

Explanation of **`while not num1.replace('.', '', 1).isdigit():`**

This line ensures that the user enters a valid numeric input (integer or decimal) before proceeding



further. x

1. `num1.replace('.', '', 1)`

- This replaces the first occurrence of . (decimal point) in the input string with an empty string (").
- The 1 argument ensures that only the first decimal point is removed, allowing valid floating-point numbers like 12.34 but rejecting multiple decimals like 12.3.4.

2. `.isdigit()`

- The `.isdigit()` method checks whether the remaining string contains only digits (0-9).
- Since we removed only one decimal point (if present), this ensures that inputs like "12.34" are considered valid, but "12.3.4" or "abc" are not.

3. while not ...

- The while not loop ensures that as long as the input is not a valid number, the program keeps prompting the user to enter a valid value.

Viva Questions:

1. What are numeric data types in Python?

Answer: Python has three numeric data types:

- `int` (Integer) → Example: `10`, `-5`
- `float` (Floating-point) → Example: `3.14`, `-2.5`
- `complex` (Complex numbers) → Example: `2 + 3j`

2. What is the difference between `int` and `float` in Python?

Answer:

- `int` represents whole numbers without a decimal point (e.g., `10`, `-3`).
- `float` represents numbers with a decimal point (e.g., `3.14`, `-2.5`).

3. How does Python handle mixed-type arithmetic operations (e.g., `int + float`)?

Answer: When an `int` and a `float` are used in an arithmetic operation, Python automatically converts the integer to a float before performing the operation.

Example: `5 + 3.2` → The result is `8.2` (float).



5. How can we check if a given input is a valid numeric value in Python?

Answer: We can use `.isdigit()` for integers and `replace('.', '', 1).isdigit()` for floating-point numbers. Example:

```
num = input("Enter a number: ")
if num.replace('.', '', 1).isdigit():
    print("Valid number")
else:
    print("Invalid number")
```

6. Why is input validation important in numeric operations?

Answer: Input validation ensures that the user enters valid numeric values, preventing errors such as `ValueError` or `ZeroDivisionError`. It helps in making the program robust and avoids unexpected crashes.

Lab Program 3: Control Structures & Decision Making

Problem Statement:

A leap year is:

- Divisible by 4 **AND**
- Not divisible by 100, **unless** it is also divisible by 400.

The program should:

1. Accept a valid year as input.
2. Check if the year is a leap year using the above conditions.
3. Display the result accordingly.
4. Ensure the input is a valid positive integer (no negative numbers, decimals, or non-numeric characters).

Objectives:

- To develop a Python program that checks whether a given year is a leap year or not, with strong input validation to ensure only valid numerical inputs are processed.

Algorithm:

1. Prompt the user to enter a year.



2. Validate the input:
 - Ensure it consists only of digits.
 - Ensure it is a positive integer.
3. Convert the validated input into an integer.
 - Apply the leap year conditions:
 - If the year is divisible by 400, it is a leap year.
 - If the year is divisible by 100 but not by 400, it is not a leap year.
 - If the year is divisible by 4 but not by 100, it is a leap year.
 - Otherwise, it is not a leap year.
4. Display the appropriate message.

Python Program:

```
1  ## Lab Program 3: Control Structures & Decision Making
2
3  # Program to check if a given year is a Leap year
4
5  # Input validation
6  year = input("Enter a valid year: ")
7
8  while not (year.isdigit() and int(year) > 0): # Ensures only positive integers
9      print("Invalid input, Please enter a positive integer")
10     year = input("Enter a valid year: ")
11
12  year = int(year) # Convert validated input to integer
13
14  # Leap year check
15  if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
16      print(year, " is a Leap Year")
17  else:
18      print(year, " is NOT a Leap Year")
```

Sample Output:

```
Enter a year: 2025
2025 is not a Leap Year
```

```
Enter a year: 2024
2024 is a Leap Year
```



```
Enter a valid year: 0
Invalid input, Please enter a positive integer
Enter a valid year: gmu
Invalid input, Please enter a positive integer
Enter a valid year: 2024.0
Invalid input, Please enter a positive integer
Enter a valid year: 2024.12
Invalid input, Please enter a positive integer
Enter a valid year: 2024
2024 is a Leap Year
```

Explanation of Input Validation:

1. `year.isdigit()`

- Checks if **all characters** in the `year` input are digits (0-9).
- Returns `True` if the input is entirely numeric.
- Returns `False` if the input contains **any non-numeric character** (e.g., letters, special characters, negative signs, or decimals).

2. `int(year) > 0`

- Converts the input `year` to an integer and checks if it is **greater than 0**.
- Ensures that the input is a **positive** number.

Why `not` is used?

The `while not (...)` makes the condition **True** when the input is **invalid**, forcing the user to re-enter a valid value.

If the input is **not a positive integer**, the loop runs again:

```
while not (year.isdigit() and int(year) > 0):
    print("Invalid input! Please enter a positive integer.")
    year = input("Enter a valid year: ")
```

This ensures the program keeps asking for input until a valid positive integer is provided.



Viva Questions:

1. What is a leap year?

A leap year is divisible by 4 but not by 100, except when also divisible by 400.

2. Which control structure is used in this program?

if-elif-else decision-making structure.

3. Why is 1900 not a leap year but 2000 is?

1900 is divisible by 100 but not by 400, while 2000 is divisible by 400.

4. How does the `int(input())` statement work?

It takes user input as a string and converts it into an integer.

5. Can this program be written using a function?

Yes, we can define a function to check leap year conditions and return results.

4. Looping Constructs & Range Iteration

Problem: Develop a Python program that generates and prints the Fibonacci sequence up to n terms, where n is provided by the user.

- Implement a **for loop** to compute and store the sequence in a list.
- Use a **while loop** to print each term dynamically.
- Allow the user to exit early by entering a negative number

Algorithm:

1. Prompt the user to enter a **positive integer (n)** for the number of terms.
2. **Validate** the input to ensure it is a positive integer. If negative, exit the program.
3. Initialize two variables, `a = 0` and `b = 1`, for Fibonacci computation.
4. Use a **for loop** to compute the Fibonacci sequence up to n terms and store them in a list.
5. Use a **while loop** to print each term dynamically.
6. If the user enters a **negative number**, exit the program early



Python Program:

```
1  ## 4. Looping Constructs & Range Iteration
2
3  # Taking input from the user with validation
4  n = input("Enter the number of terms for Fibonacci sequence: ")
5
6  # Validate input to ensure it is a positive integer
7  while not (n.isdigit() and int(n) > 0):
8      print("Invalid input, Please enter a positive integer.")
9      n = input("Enter the number of terms to generate in the Fibonacci sequence: ")
10
11 n = int(n) # Convert valid input to integer
12
13 a = 0
14 b = 1
15 # a, b = 0, 1
16 fibonacci_sequence = []
17
18 # Generate Fibonacci sequence using a for loop
19 # for _ in range(n):
20 for i in range(n):
21     fibonacci_sequence.append(a)
22     # a, b = b, a + b
23     a = b
24     b = a + b
25
26 print("\nFibonacci Sequence:")
27
28 # Print each term dynamically using a while loop
29 index = 0
30 while index < len(fibonacci_sequence):
31     print(fibonacci_sequence[index], end="\t")
32     index += 1
33     # index = index + 1
```

Enter the number of terms for Fibonacci sequence: 6

Fibonacci Sequence:

0 1 2 4 8 16



```
Enter the number of terms for Fibonacci sequence: -9
Invalid input, Please enter a positive integer.
Enter the number of terms to generate in the Fibonacci sequence: 0
Invalid input, Please enter a positive integer.
Enter the number of terms to generate in the Fibonacci sequence: gmu
Invalid input, Please enter a positive integer.
Enter the number of terms to generate in the Fibonacci sequence: 4

Fibonacci Sequence:
0      1      2      3
```

Explanation:

The `append()` function in Python is used to add an element to the end of a list.

- **Syntax:** `list.append(element)`
- **Effect:** Adds `element` to the end of the `list`.

```
while index < len(fibonacci_sequence):
```

This is a **while loop condition** that ensures iteration continues as long as the `index` variable is **less than** the length of the `fibonacci_sequence` list.

1. `len(fibonacci_sequence)`:

- This function returns the total number of elements in the `fibonacci_sequence` list.

2. `index < len(fibonacci_sequence)`:

- The loop runs as long as `index` is a valid position inside the list.
- If `index` reaches `len(fibonacci_sequence)`, the condition becomes `False`, and the loop stops.

Viva Questions:

1. What is the Fibonacci sequence?

Answer: The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1. Example: 0, 1, 1, 2, 3, 5, 8, ...



2. Why do we use a for loop to generate the Fibonacci sequence?

Answer: The for loop is used because the number of terms (n) is known in advance, making it an efficient way to iterate and generate the sequence.

3. What is the purpose of the while loop in this program?

Answer: The while loop is used to print each term dynamically, ensuring that all elements of the generated Fibonacci sequence are displayed one by one.

4. How does the program handle early exit when a negative number is entered?

Answer: If the user inputs a negative number, the program detects it and exits immediately by printing a message like "Exiting program...".

5. Why do we use append() in the Fibonacci program?

Answer: The append() function is used to add new Fibonacci numbers to the list dynamically as the sequence is generated.

6. What happens if the user enters n = 1?

Answer: If n = 1, the program will generate and print only the first term of the Fibonacci sequence, which is [0].

5. List & Tuple Operations

Objective:

To develop a Python program that manages a student database using lists and tuples, allowing users to add, search, remove student records, and calculate the average marks.

Problem: Create a Python program that manages a **student database** using lists and tuples.

- Use a list to store multiple student records, where each student is represented as a tuple containing (USN, Name, Marks).
- Implement functions to:
 - **Add a student record**



- **Search for a student by USN**
- **Calculate and display the average marks**
- **Remove a student record**

Algorithm:

1. Create an **empty list** to store student records.
2. Define a function to **add a student**
3. Define a function to **search for a student by USN** and return their details if found.
4. Define a function to **calculate the average marks** of all students in the database.
5. Define a function to **remove a student record** using their USN.
6. Define a function to **Display all students record**
7. Implement a loop-based **menu-driven approach** to allow users to interact with the system until they choose to exit

Python Program:

```
1  ## 5. List & Tuple Operations
2
3  # Student Database Management using List & Tuples
4
5  # List to store student records (each student is a tuple)
6  students_list = []
7
8  # Function to add a student record
9  def add_student():
10     usn = input("Enter Student USN: ")
11
12     name = input("Enter Student Name: ")
13
14     while True:
15         marks = int(input("Enter Marks (0-100): "))
16         if 0 <= marks <= 100:
17             break
18         print("Invalid input, Marks must be between 0 and 100.")
19
20     students_list.append((usn, name, marks))
21     print("Student added successfully\n")
22
```

```
23 # Function to search for a student by USN
24 def search_student():
25     usn = input("Enter Student USN to search: ")
26     for student in students_list:
27         if student[0] == usn:
28             print("Student Found:")
29             print("USN =", student[0])
30             print("Name =", student[1])
31             print("Marks =", student[2])
32             return
33     print("Student not found\n")
34
35 # Function to calculate average marks
36 def calculate_average():
37     if len(students_list) == 0:
38         print("No students in the database to calculate average\n")
39         return
40
41     total_marks = sum(student[2] for student in students_list)
42     avg_marks = total_marks / len(students_list)
43     print("Average Marks of Students: ", avg_marks, "\n")
44
45 # Function to remove a student record
46 def remove_student():
47     remove_id = input("Enter Student USN to remove: ")
48     for student in students_list:
49         if student[0] == remove_id:
50             students_list.remove(student)
51             print("Student record removed successfully\n")
52             return
53     print("Student USN not found\n")
54
55 # Function to display all student records
56 def display_students():
57     if len(students_list) == 0:
58         print("No student records available\n")
59         return
60
61     print("\nStudent Records:")
62     print("USN\t\tName\t\tMarks")
63     print("-" * 30)
64     for student in students_list:
65         print(f"{student[0]}\t\t{student[1]}\t\t{student[2]}")
66     print()
```



```
68 # Menu-driven approach
69 while True:
70     print("Student Database Management System")
71     print("1. Add Student")
72     print("2. Search Student by USN")
73     print("3. Calculate Average Marks")
74     print("4. Remove Student")
75     print("5. Display All Students")
76     print("6. Exit")
77
78     choice = input("Enter your choice (1-6): ")
79
80     if choice == "1":
81         add_student()
82     elif choice == "2":
83         search_student()
84     elif choice == "3":
85         calculate_average()
86     elif choice == "4":
87         remove_student()
88     elif choice == "5":
89         display_students()
90     elif choice == "6":
91         print("Exiting program...")
92         break
93     else:
94         print("Invalid choice! Please enter a number between 1 and 6\n")
```

Student Database Management System

1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit

Enter your choice (1-6): 1

Enter Student USN: AI001

Enter Student Name: Krishna

Enter Marks (0-100): 67

Student added successfully



Student Database Management System

1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit

Enter your choice (1-6): 1

Enter Student USN: CS002

Enter Student Name: Raghu

Enter Marks (0-100): 89

Student added successfully

Student Database Management System

1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit

Enter your choice (1-6): 3

Average Marks of Students: 78.0

Student Database Management System

1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit

Enter your choice (1-6): 5

Student Records:

USN	Name	Marks
-----	------	-------

AI001	Krishna	67
-------	---------	----

CS002	Raghu	89
-------	-------	----

Student Database Management System

1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit

Enter your choice (1-6): 4

Enter Student USN to remove: 001

Student USN not found



Srishyla Education Trust ®

GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)

Post Box no-4, PB Road, Davangere-577006

FACULTY OF ENGINEERING AND TECHNOLOGY

School of Computer Science and Technology

```
Student Database Management System
1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit
Enter your choice (1-6): 4
Enter Student USN to remove: CS002
Student record removed successfully
```

```
Student Database Management System
1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit
Enter your choice (1-6): 5
```

```
Student Records:
USN          Name          Marks
-----
AI001   Krishna 67
```

```
Student Database Management System
1. Add Student
2. Search Student by USN
3. Calculate Average Marks
4. Remove Student
5. Display All Students
6. Exit
Enter your choice (1-6): 6
Exiting program...
```

Explanation:

`len(students_list)` is a function call that returns the number of elements (length) in the list `students_list`.

- `students_list` is a Python list that contains student records.
- `len(students_list)` counts how many elements (students) are present in the list and returns that count as an integer.



```
students_list = ["Alice", "Bob", "Charlie"]  
print(len(students_list)) # Output: 3
```

```
total_marks = sum(student[2] for student in students_list)
```

This line calculates the sum of all students' marks in the **students_list**

- `students_list` is a list of student records, where each student is stored as a tuple (usn, name, marks).
- `student[2]` extracts the **marks** from each tuple.
- This part generates a sequence (or generator) of marks.

`sum()` Function

- `sum()` takes this sequence of marks and adds them together.
- The result is stored in `total_marks`.

```
students_list = [  
    ("1BM22CS001", "Alice", 85),  
    ("1BM22CS002", "Bob", 90),  
    ("1BM22CS003", "Charlie", 78)  
]  
  
total_marks = sum(student[2] for student in students_list)  
print("Total Marks:", total_marks)  
# Output: Total Marks: 253
```

Viva Questions:

1. How does the `sum()` function work in the program?

Answer: The `sum()` function is used to calculate the total marks of all students by iterating over the list of tuples and summing the marks stored at index `2` in each tuple.

2. What data structure is used to store student records, and why?

Answer: A list of tuples is used because each student record consists of multiple attributes (USN, name, marks), and tuples ensure that the data remains structured and immutable.



3. Why do we validate marks before adding them to the list?

Answer: Validation ensures that the marks entered by the user are numeric and within the valid range (0-100). This prevents incorrect or invalid data from being stored in the system.

4. What happens if a student with a given USN is not found in the `search_student()` function?

Answer: If the USN is not found, the function prints "Student not found!" and returns without displaying any record.

5. How does the `remove_student()` function work, and what issue might occur if the ID is not found?

Answer: It iterates through the list, searches for the student by USN, and removes the matching tuple. If the USN is not found, it prints "Student ID not found!", but the issue is that the message is inside the loop, so it might print multiple times.

6. How would you modify the program to display all student records?

Answer: A new function can be added that iterates through `students_list` and prints each student's details in a structured format.

6. Dictionary & Set Operations

Problem: Write a Python program to perform operations on an **employee database** using dictionaries and sets. Implementing various operations.

- Store employee details as key-value pairs (EmployeeID: Name).
- Implement functionalities to:
 - **Add and remove employees**
 - **Search for an employee by ID**
 - **Display all employee names in sorted order**
- Use sets to manage employee **departments**, performing
 - **Union** operation.
 - **Intersection** operation.
 - **Difference** operation.

Algorithm:

1. Initialize an empty dictionary **employee** to store employee records.
2. Define functions for:

- Adding an employee.
 - Removing an employee.
 - Searching for an employee.
 - Displaying all employees in **sorted order**.
 - Performing **set operations** on departments.
3. Use sets **department_A** and **department_B** to represent employees in different departments.
 4. Provide a **menu-driven** system for user interaction.
 5. Validate inputs to prevent duplicate entries and incorrect data.
 6. Exit the program when the user selects the exit option.

Python Program:

```
1  ## 6. Dictionary & Set Operations
2  # Dictionary to store employee details (EmployeeID: Name)
3
4  employees = {}    #Empty Distctionary
5
6  # Sets to store departments
7  department_CSE = {"E101", "E102", "E103", "E104"}
8  department_AIML = {"E103", "E104", "E105", "E106"}
9
10 # Function to add an employee
11 def add_employee():
12     emp_id = input("Enter Employee ID: ")
13
14     if emp_id in employees:
15         print("Employee ID already exists!\n")
16         return
17
18     emp_name = input("Enter Employee Name: ")
19     employees[emp_id] = emp_name
20     print("Employee ", emp_name, " added successfully")
21
```



```
22 # Function to remove an employee
23 def remove_employee():
24     emp_id = input("Enter Employee ID to remove: ")
25
26     if emp_id in employees:
27         del employees[emp_id]
28         print(f"Employee with ID {emp_id} removed successfully\n")
29     else:
30         print("Employee ID not found\n")
31
32 # Function to search for an employee
33 def search_employee():
34     emp_id = input("Enter Employee ID to search: ")
35
36     if emp_id in employees:
37         print(f"Employee Found: {emp_id} -> {employees[emp_id]}\n")
38     else:
39         print("Employee ID not found")
40
41 # Function to display all employees sorted by name
42 def display_sorted_employees():
43     if not employees:
44         print("No employees in the database")
45         return
46
47     print("Employees sorted by name:")
48     for emp_id, emp_name in sorted(employees.items(), key=lambda x: x[1]):
49         print(f"{emp_id}: {emp_name}")
50     print()
51
52 # Function to perform set operations on departments
53 def perform_set_operations():
54     print("\nSet Operations on Departments:")
55     print(f"Department A: {department_CSE}")
56     print(f"Department B: {department_AIML}")
57
58     print("\nUnion (Employees in either A or B):", department_CSE | department_AIML)
59     print("Intersection (Employees in both A and B):", department_CSE & department_AIML)
60     print("Difference (Employees in A but not in B):", department_CSE - department_AIML)
61     print("Difference (Employees in B but not in A):", department_CSE - department_AIML, "\n")
```

```
63 # Menu-driven approach
64 while True:
65     print("Employee Database Management System")
66     print("1. Add Employee")
67     print("2. Remove Employee")
68     print("3. Search Employee")
69     print("4. Display Employees in Sorted Order")
70     print("5. Perform Set Operations on Departments")
71     print("6. Exit")
72
73     choice = input("Enter your choice (1-6): ")
74
75     if choice == "1":
76         add_employee()
77     elif choice == "2":
78         remove_employee()
79     elif choice == "3":
80         search_employee()
81     elif choice == "4":
82         display_sorted_employees()
83     elif choice == "5":
84         perform_set_operations()
85     elif choice == "6":
86         print("Exiting program...")
87         break
88     else:
89         print("Invalid choice, Please enter a number between 1 and 6\n")
```

```
Employee Database Management System
1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

Enter your choice (1-6): 1
Enter Employee ID: E101
Enter Employee Name: Karuna
Employee Karuna added successfully
```

```
Employee Database Management System
1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

Enter your choice (1-6): 1
Enter Employee ID: E101
Employee ID already exists!
```



Srishyla Education Trust ®

GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)

Post Box no-4, PB Road, Davangere-577006

FACULTY OF ENGINEERING AND TECHNOLOGY

School of Computer Science and Technology

Employee Database Management System

1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

Enter your choice (1-6): 4

Employees sorted by name:

E102: Baanu

E101: Karuna

E103: Pruthvi

Employee Database Management System

1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

Enter your choice (1-6): 2

Enter Employee ID to remove: 4

Employee ID not found

Employee Database Management System

1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

Enter your choice (1-6): 3

Enter Employee ID to search: E101

Employee Found: E101 -> Karuna



```
Employee Database Management System
```

1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

```
Enter your choice (1-6): 5
```

```
Set Operations on Departments:
```

```
Department A: {'E101', 'E103', 'E104', 'E102'}
```

```
Department B: {'E103', 'E106', 'E104', 'E105'}
```

```
Union (Employees in either A or B): {'E103', 'E105', 'E101', 'E102', 'E106', 'E104'}
```

```
Intersection (Employees in both A and B): {'E103', 'E104'}
```

```
Difference (Employees in A but not in B): {'E101', 'E102'}
```

```
Difference (Employees in B but not in A): {'E101', 'E102'}
```

```
Employee Database Management System
```

1. Add Employee
2. Remove Employee
3. Search Employee
4. Display Employees in Sorted Order
5. Perform Set Operations on Departments
6. Exit

```
Enter your choice (1-6): 6
```

```
Exiting program...
```

Explanation:

`del` Statement in Python

The `del` statement in Python is used to delete objects, variables, or elements from collections (like lists, dictionaries, and sets).

1. Deleting a Variable

```
x = 10
print(x) # Output: 10
del x
print(x) # Error: NameError: name 'x' is not defined
```



- The variable `x` is deleted using `del x`.
- Trying to access `x` afterward results in an error since it no longer exists.

2. Explanation of the Line

```
for emp_id, emp_name in sorted(employees.items(), key=lambda x: x[1]):
```

This line is used to sort and iterate over employee records stored in the employees dictionary based on employee names in ascending order

1. `employees.items()`

- `employees` is a dictionary where Employee ID is the key, and Employee Name is the value.
- `employees.items()` returns a list of key-value pairs (tuples).

Example:

```
employees = {  
    "E102": "Baanu",  
    "E101": "Karuna",  
    "E103": "Pruthvi"  
}
```

- `employees.items()` → `[("E102", "Baanu"), ("E101", "Karuna"), ("E103", "Pruthvi")]`

2. `sorted(employees.items(), key=lambda x: x[1])`

- `sorted()` is used to sort the list of tuples.
- `key=lambda x: x[1]` means sorting is done based on the **second element** (`x[1]`) of each tuple, which is the employee name.
- Sorting based on names:

- **Original List:**

```
[("E102", "Baanu"), ("E101", "Karuna"), ("E103", "Pruthvi")]
```

- **Sorted Order (by name):**

```
[("E102", "Baanu"), ("E101", "Karuna"), ("E103", "Pruthvi")]
```

(Here, it's already sorted, but if it were shuffled, this would reorder them.)



3. `for emp_id, emp_name in sorted(...)`

- The `for` loop extracts each tuple into two variables:

```
for emp_id, emp_name in sorted(employees.items(), key=lambda x: x[1]):  
    print(emp_id, emp_name)
```

- Iterates over sorted employee records:
 - First iteration:** `emp_id = "E102"`, `emp_name = "Baanu"`
 - Second iteration:** `emp_id = "E101"`, `emp_name = "Karuna"`
 - Third iteration:** `emp_id = "E103"`, `emp_name = "Pruthvi"`

Viva Questions:

1. What is the difference between a dictionary and a set in Python?

Answer: A dictionary stores **key-value pairs**, whereas a set stores **only unique elements** without any key-value mapping.

2. How do you add and remove elements in a dictionary?

Answer:

- To add: `dictionary[key] = value`
- To remove: `del dictionary[key]` or `dictionary.pop(key)`

3. What are the different set operations available in Python?

Answer:

- Union (`|`):** Combines elements from both sets.
- Intersection (`&`):** Finds common elements.
- Difference (`-`):** Finds elements in one set but not in the other.

4. How do you sort a dictionary by values in Python?

Answer: Using `sorted(dictionary.items(), key=lambda x: x[1])` sorts the dictionary by values.



5. How do you check if a key exists in a dictionary?

Answer: Use the `in` keyword:

```
if key in dictionary:  
    print("Key exists")
```

6. What happens if you try to remove a key that does not exist in a dictionary?

Answer: Python raises a `KeyError`, unless we use `dictionary.pop(key, default_value)`.

7. Arrays & NumPy Operations

Problem: Implement a Python program that demonstrates **array operations** using NumPy. (Student Marks Analysis)

Objective:

To demonstrate basic NumPy operations such as element-wise arithmetic, aggregation functions, and slicing using student marks as a real-time example.

Problem Statement:

A class has two test scores recorded for each student. Implement a Python program using NumPy to:

- Store the marks of students in NumPy arrays.
- Perform element-wise operations (addition, subtraction, multiplication, and division) between two test scores.
- Compute aggregate values like total marks, average marks, highest, and lowest scores.
- Extract specific marks using slicing.

Algorithm:

1. **Import NumPy.**
2. **Take user input to create two NumPy arrays** representing marks of students in Test 1 and Test 2.
3. **Perform element-wise arithmetic operations:** addition, subtraction, multiplication, and division.
4. **Compute aggregate values:** sum, mean, max, and min.
5. **Use slicing** to extract specific sections of the array (e.g., first three students' marks).
6. **Display the results.**

Python Program:

```
1  ## 7. Arrays & NumPy Operations
2  import numpy as np
3
4  # Creating NumPy arrays for student marks
5  test1_marks = np.array([85, 78, 92, 67, 88]) # Marks of students in Test 1
6  test2_marks = np.array([80, 75, 95, 70, 85]) # Marks of students in Test 2
7
8  # Perform element-wise arithmetic operations
9  total_marks = test1_marks + test2_marks
10 difference_marks = test1_marks - test2_marks
11 product_marks = test1_marks * test2_marks
12 division_marks = test1_marks / test2_marks |
13
14 # Compute sum, mean, max, and min
15 sum_test1 = np.sum(test1_marks)
16 mean_test1 = np.mean(test1_marks)
17 max_test1 = np.max(test1_marks)
18 min_test1 = np.min(test1_marks)
19
20 # Slicing - Extract first three students' marks
21 first_three_test1 = test1_marks[:3]
22 # Extract last two students' marks
23 last_two_test1 = test1_marks[-2:]
24 # Extract alternate students' marks (every second student)
25 alternate_test1 = test1_marks[::2]
26 # Extract middle three students' marks (excluding first and last)
27 middle_three_test1 = test1_marks[1:4]
28
29 # Display the results of : Perform element-wise arithmetic operations
30 print("Test 1 Marks: ", test1_marks)
31 print("Test 2 Marks: ", test2_marks)
32 print("\nTotal Marks (Element-wise Addition):", total_marks)
33 print("Difference of Marks (Element-wise Subtraction):", difference_marks)
34 print("Product of Marks (Element-wise Multiplication):", product_marks)
35 print("Division of Marks (Element-wise Division):", division_marks)
36
37 # Display the results of : Compute sum, mean, max, and min
38 print("\nSum of Test 1 Marks:", sum_test1)
39 print("Average Marks in Test 1:", mean_test1)
40 print("Highest Marks in Test 1:", max_test1)
41 print("Lowest Marks in Test 1:", min_test1)
```



```
43 # Display the slicing results
44 print("\nFirst Three Students' Marks in Test 1:", first_three_test1)
45 print("Last Two Students' Marks in Test 1:", last_two_test1)
46 print("Alternate Students' Marks in Test 1:", alternate_test1)
47 print("Middle Three Students' Marks in Test 1:", middle_three_test1)
```

Test 1 Marks: [85 78 92 67 88]

Test 2 Marks: [80 75 95 70 85]

Total Marks (Element-wise Addition): [165 153 187 137 173]

Difference of Marks (Element-wise Subtraction): [5 3 -3 -3 3]

Product of Marks (Element-wise Multiplication): [6800 5850 8740 4690 7480]

Division of Marks (Element-wise Division): [1.0625 1.04 0.96842105 0.95714286 1.03529412]

Sum of Test 1 Marks: 410

Average Marks in Test 1: 82.0

Highest Marks in Test 1: 92

Lowest Marks in Test 1: 67

First Three Students' Marks in Test 1: [85 78 92]

Last Two Students' Marks in Test 1: [67 88]

Alternate Students' Marks in Test 1: [85 92 88]

Middle Three Students' Marks in Test 1: [78 92 67]

Explanation:

The `sum()` function calculates the total of all elements, `mean()` computes the average value, `max()` returns the highest element, and `min()` returns the lowest element in a NumPy array.

Element-wise arithmetic operations in NumPy allow you to perform mathematical operations directly on corresponding elements of arrays. Unlike lists in Python, NumPy arrays support vectorized operations, making calculations faster and more efficient.

Explanation of Slicing Operations:

1. `[ :3 ]` → Extracts the **first three** elements from the array.
2. `[ -2: ]` → Extracts the **last two** elements from the array.
3. `[ ::2 ]` → Extracts **every second** element (starting from the first).
4. `[ 1:4 ]` → Extracts elements from index **1 to 3** (excluding index 4).



Viva Questions on NumPy Array Operations

1. What is NumPy, and why is it used in Python?

Answer: NumPy (Numerical Python) is a powerful library for numerical computations in Python. It provides support for large, multi-dimensional arrays and matrices, along with mathematical functions to operate on them efficiently.

2. How do you create a NumPy array from a Python list?

Answer: We can use the `numpy.array()` function to convert a Python list into a NumPy array.

Example:

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr)
```

3. What is the difference between a Python list and a NumPy array?

Answer:

- A Python **list** can store elements of different data types, whereas a NumPy **array** requires all elements to be of the same type.
- NumPy arrays support **vectorized operations** (fast element-wise operations), while Python lists require loops for such operations.

4. Explain element-wise arithmetic operations in NumPy.

Answer: NumPy allows operations like addition, subtraction, multiplication, and division to be performed directly between arrays, element by element.

Example:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b) # Output: [5 7 9]
```



5. How can you find the sum, mean, maximum, and minimum of a NumPy array?

Answer: NumPy provides built-in functions:

- `np.sum(arr)` : Returns the sum of all elements.
- `np.mean(arr)` : Returns the mean (average) value.
- `np.max(arr)` : Returns the maximum value.
- `np.min(arr)` : Returns the minimum value.

6. What is slicing in NumPy, and how is it used?

Answer: Slicing is used to extract specific parts of an array. It follows the syntax

`array[start:stop:step]`.

Example:

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4]) # Output: [20 30 40]
```

7. How does NumPy handle division by zero?

Answer: If an element in an array is divided by zero, NumPy does not crash but returns `inf` (infinity) instead of an error.

Example:

```
arr = np.array([10, 20, 0])
print(arr / arr) # Output: [1. 1. nan]
```

8. Can we perform mathematical operations between a NumPy array and a scalar value?

Answer: Yes, NumPy supports **broadcasting**, allowing operations between an array and a scalar.

Example:

```
arr = np.array([1, 2, 3])
print(arr * 2) # Output: [2 4 6]
```

8. Pandas for Tabular Data Processing

Problem: Develop a Python program that reads and processes a CSV file containing **student marks** using Pandas.

- Read the dataset into a Pandas **DataFrame**.

- Display the **top 5 and bottom 5 rows** of the dataset.
- Filter and display students who scored above 80 marks.
- Compute the **average, highest, and lowest** marks per subject.
- Write the processed data into a new CSV file.

Algorithm:

1. **Import the required library** – Import the Pandas library to handle tabular data.
2. **Read the CSV file** – Load student marks data into a Pandas DataFrame using `pd.read_csv()`.
3. **Display the dataset preview** – Use `.head(5)` to display the top 5 rows and `.tail(5)` to display the bottom 5 rows.
4. **Filter students scoring above 80** – Apply a condition to filter and display students with marks greater than 80.
5. **Compute statistical measures** – Use `.mean()`, `.max()`, and `.min()` functions to calculate the average, highest, and lowest marks for each subject.
6. **Save processed data to a new CSV file** – Write the filtered or processed DataFrame to a new CSV file using `.to_csv()`.

Python Program:

```
1  ## 8. Pandas for Tabular Data Processing
2  import pandas as pd
3
4  # Step 1: Read the dataset from a CSV file
5  df = pd.read_csv("Desktop/Asha/Student_Marks.csv")
6
7  # Step 2: Display top 5 and bottom 5 rows
8  print("Top 5 Rows:\n", df.head(5))
9  print("\nBottom 5 Rows:\n", df.tail(5))
10
11 # Filter students who scored above 80 marks in Maths
12 high_scorers = df[df["Maths"] > 80]
13 print("\nStudents scoring above 80 in Maths:\n", high_scorers)
14
15 # Filter students who scored above 80 marks in Physics
16 # high_scorers = df[df["Physics"] > 80]
17 # ("\nStudents scoring above 80 in Maths:\n", high_scorers)
```



```
19 # Filter students who scored above 80 marks in Chemistry
20 # high_scorers = df[df["Chemistry"] > 80]
21 # print("\nStudents scoring above 80 in Maths:\n", high_scorers)
22
23
24 # Filter students who scored above 80 marks in Biology
25 # high_scorers = df[df["Biology"] > 80]
26 # print("\nStudents scoring above 80 in Maths:\n", high_scorers)
27
28 # Compute average, highest, and lowest marks per subject
29 print("\nAverage Marks per Subject:\n", df.iloc[:, 2:].mean())
30 print("\nHighest Marks per Subject:\n", df.iloc[:, 2:].max())
31 print("\nLowest Marks per Subject:\n", df.iloc[:, 2:].min())
32
33 # Write the processed data into a new CSV file
34 high_scorers.to_csv("Desktop/Asha/high_scorers.csv", index=False)
35 print("\nProcessed data saved to 'high_scorers.csv'.") |
```

Top 5 Rows:

	Unnamed: 0	Gender	Maths	Physics	Chemistry	English	Biology
0	John	M	55	45	56	45	21
1	Suresh	M	75	96	78	64	90
2	Ramesh	M	25	54	89	32	95
3	Jessica	F	78	96	86	6	54
4	Jennifer	F	58	96	78	46	96

Bottom 5 Rows:

	Unnamed: 0	Gender	Maths	Physics	Chemistry	English	Biology
5	Annu	F	45	87	52	45	55
6	pooja	F	55	64	61	34	75
7	Ritesh	M	54	76	87	60	25
8	Farha	F	55	63	89	23	78
9	Mukesh	M	96	46	77	45	58

Students scoring above 80 in Maths:

	Unnamed: 0	Gender	Maths	Physics	Chemistry	English	Biology
9	Mukesh	M	96	46	77	45	58

Average Marks per Subject:

Maths	59.6
Physics	72.3
Chemistry	75.3
English	40.0
Biology	64.7
dtype:	float64



Highest Marks per Subject:

Maths	96
Physics	96
Chemistry	89
English	64
Biology	96

dtype: int64

Lowest Marks per Subject:

Maths	25
Physics	45
Chemistry	52
English	6
Biology	21

dtype: int64

Processed data saved to 'high_scorers.csv'.

Explanation:

In Pandas, `df.head(n)` and `df.tail(n)` are used to display specific portions of a DataFrame:

1. `df.head(5)` – Display the First 5 Rows

- The `.head(5)` function retrieves and prints the first 5 rows of the DataFrame `df`.
- This helps in getting an overview of the dataset structure, including column names, data types, and sample data.
- If the dataset has fewer than 5 rows, it will display all available rows.

2. `df.tail(5)` – Display the Last 5 Rows

- The `.tail(5)` function retrieves and prints the last 5 rows of the DataFrame `df`.
- This helps in checking data consistency and verifying the structure at the end of the dataset.
- If the dataset has fewer than 5 rows, it will display all available rows.

The statement `df.iloc[:, 2:]` is used for row-column selection in a Pandas DataFrame using `.iloc[]`, which allows integer-based indexing.

- `df.iloc` → Accesses rows and columns using index positions.
- `:` → Means "select all rows."
- `2:` → Means "select columns starting from index 2 (third column) to the last column."

`index=False` in `to_csv()`

In the Pandas function `.to_csv()`, the parameter `index=False` is used to prevent writing the index column to the output CSV file.

- If the dataset has fewer than 5 rows, it will display all available rows.

Index column is not added to "high_scorers.csv" file

Unnamed: 0	Gender	Maths	Physics	Chemistry	English	Biology	Maths_Gr	Physics_Gr	Chemistry_Gr	English_Gr	Biology_Gr	Grade
John	M	55	45	56	45	21	B	C	B	C	D	
Suresh	M	75	96	78	64	90	A+	A+	A+	A	A+	
Ramesh	M	25	54	89	32	95	D	B	A+	D	A+	
Jessica	F	78	96	86	6	54	A+	A+	A+	D	B	
Jennifer	F	58	96	78	46	96	B	A+	A+	C	A+	
Annu	F	45	87	52	45	55	C	A+	B	C	B	
pooja	F	55	64	61	34	75	B	A	A	D	A+	
Ritesh	M	54	76	87	60	25	B	A+	A+	A	D	
Farha	F	55	63	89	23	78	B	A	A+	D	A+	
Mukesh	M	96	46	77	45	58	A+	C	A+	C	B	

Viva Questions:

1. What is Pandas, and why is it used in data processing?

Answer: Pandas is a Python library used for data manipulation and analysis. It provides data structures like Series and DataFrame, making it easy to work with structured data such as CSV files.

2. How do you read a CSV file into a Pandas DataFrame?

Answer: We use the `pd.read_csv("filename.csv")` function to load a CSV file into a Pandas DataFrame.

3. What does the `head()` function do in Pandas?

Answer: The `head()` function displays the first five rows of a DataFrame by default. We can specify the number of rows to display, e.g., `df.head(3)`.

4. How can you filter rows in a Pandas DataFrame based on a condition?

Answer: We can use boolean indexing. For example, to filter students who scored above 80 marks:

```
df[df["Marks"] > 80]
```




5. What are some key statistical functions available in Pandas for data analysis?

Answer: Pandas provides various statistical functions such as:

- `df.mean()` – Computes the average of values in each column.
- `df.max()` – Finds the maximum value in each column.
- `df.min()` – Finds the minimum value in each column.
- `df.describe()` – Provides summary statistics.

6. How do you write a Pandas DataFrame back to a CSV file?

Answer: We use the `to_csv()` function. Example:

```
df.to_csv("output.csv", index=False)
```

The `index=False` parameter ensures that the index is not saved in the CSV file.

9. Data Visualization with Matplotlib

Problem: Write a Python program that visualizes **sales data** using Matplotlib.

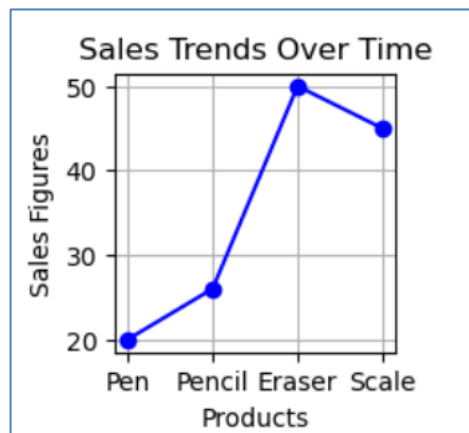
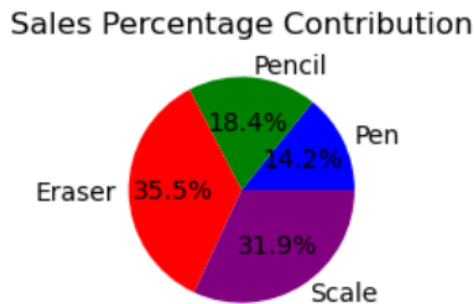
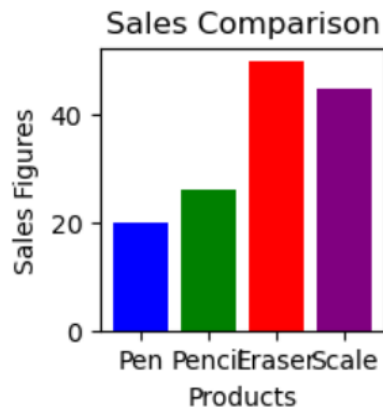
- Accept sales data (product names and sales figures) from the user.
- Generate the following visualizations:
 - **Bar chart** for sales comparison.
 - **Pie chart** showing percentage contribution of each product.
 - **Line plot** depicting sales trends over time.
- Customize plots with titles, labels, legends, and colors.
- Export the graphs as images.

Algorithm:

1. Accept user input for product names and sales figures.
2. Store the data in appropriate Python data structures (lists or dictionaries).
3. Generate visualizations using Matplotlib:
 - **Bar Chart:** Compare sales figures for different products.
 - **Pie Chart:** Show percentage contribution of each product.
 - **Line Plot:** Depict sales trends over time.
4. Customize plots by adding titles, axis labels, legends, and colors.
5. Export graphs as images using `plt.savefig()`.

```
1  ## 9. Data Visualization with Matplotlib
2  import matplotlib.pyplot as plt
3
4  # Accept sales data from the user
5  num_products = int(input("Enter the number of products: "))
6  products = []
7  sales = []
8
9  for i in range(num_products):
10     product_name = input(f"Enter name of product {i+1}: ")
11     sales_figure = float(input(f"Enter sales figure for {product_name}: "))
12     products.append(product_name)
13     sales.append(sales_figure)
14
15 # Generate Bar Chart for sales comparison
16 plt.figure(figsize=(2, 2))
17 plt.bar(products, sales, color=['blue', 'green', 'red', 'purple', 'orange'])
18 plt.xlabel("Products")
19 plt.ylabel("Sales Figures")
20 plt.title("Sales Comparison")
21
22 plt.savefig("bar_chart.png") # Export the bar chart as an image
23 plt.show()
24
25 # Generate Pie Chart for percentage contribution
26 plt.figure(figsize=(2, 2))
27 plt.pie(sales, labels=products, autopct='%1.1f%%', colors=['blue', 'green',
28                                                         'red', 'purple', 'orange'])
29 plt.title("Sales Percentage Contribution")
30 plt.savefig("pie_chart.png") # Export the pie chart as an image
31 plt.show()
32
33 # Generate Line Plot for sales trends
34 plt.figure(figsize=(2, 2))
35 plt.plot(products, sales, marker='o', linestyle='-', color='blue')
36 plt.xlabel("Products")
37 plt.ylabel("Sales Figures")
38 plt.title("Sales Trends Over Time")
39 plt.grid()
40 plt.savefig("line_chart.png") # Export the line chart as an image
41 plt.show()
```

```
Enter the number of products: 4
Enter name of product 1: Pen
Enter sales figure for Pen: 20
Enter name of product 2: Pencil
Enter sales figure for Pencil: 26
Enter name of product 3: Eraser
Enter sales figure for Eraser: 50
Enter name of product 4: Scale
Enter sales figure for Scale: 45
```



Explanation of Inbuilt Functions Used:

1. `import matplotlib.pyplot as plt` → Imports Matplotlib's Pyplot module for visualization.
2. `input()` → Accepts user input as a string.
3. `split(',')` → Splits the input string into a list based on commas.
4. `map(int, input().split(','))` → Converts the list of sales figures from string to integer.
5. `plt.figure(figsize=(6,4))` → Creates a new figure of given size (width 6, height 4).
6. `plt.bar(x, y, color='blue')` → Creates a bar chart with x-axis as products and y-axis as sales.
7. `plt.xlabel("Label")` → Sets the label for the x-axis.



8. `plt.ylabel("Label")` → Sets the label for the y-axis.
9. `plt.title("Title")` → Sets the title of the chart.
10. `plt.savefig("filename.png")` → Saves the current plot as an image file.
11. `plt.show()` → Displays the plot.
12. `plt.pie(values, labels=labels, autopct='%1.1f%%')` → Creates a pie chart with labels and percentage display.
13. `plt.plot(x, y, marker='o', linestyle='-', color='purple')` → Plots a line graph with markers and color.
14. `plt.grid(True)` → Enables grid lines for better readability.

Viva Questions:

1. What is Matplotlib?

- Matplotlib is a Python library used for creating static, animated, and interactive visualizations.

2. What is the purpose of `plt.figure(figsize=(6,4))` ?

- It creates a new figure with a specified width and height in inches.

3. How do you create a bar chart using Matplotlib?

- Using `plt.bar(x, y, color='blue')`, where `x` represents categories and `y` represents values.

4. What does `autopct='%1.1f%%'` do in a pie chart?

- It displays percentage values with one decimal place inside the pie chart slices.

5. How can you add labels to the x-axis and y-axis in a plot?

- Using `plt.xlabel("X-axis label")` and `plt.ylabel("Y-axis label")`.

6. What is the purpose of `plt.grid(True)` in a line chart?

- It enables grid lines for better readability of data points.

7. How can you save a plot as an image?

- Using `plt.savefig("filename.png")`.



Srishyla Education Trust ®

GM UNIVERSITY

(Established under the Karnataka State Act No. 19 of 2023)

Post Box no-4, PB Road, Davangere-577006

FACULTY OF ENGINEERING AND TECHNOLOGY

School of Computer Science and Technology

8. What is the difference between `plt.show()` and `plt.savefig()` ?

- `plt.show()` displays the plot, while `plt.savefig()` saves the plot as an image file.

9. How do you plot multiple graphs in a single figure?

- Using `plt.subplot(rows, cols, index)` to create multiple subplots in one figure.

10. Which Python library is commonly used with Matplotlib for data handling?

- Pandas is commonly used for data handling before visualization with Matplotlib.

10. Combined Project: Exploratory Data Analysis (EDA) on a Real Dataset

Problem:

Perform **Exploratory Data Analysis (EDA)** on a dataset (e.g., population, COVID-19 statistics, stock market data) using Pandas, NumPy, and Matplotlib.

- **Load** and display key statistics of the dataset.
- **Clean and preprocess** data by handling missing values.
- **Visualize trends and distributions** using scatter plots, histograms, and bar charts.
- **Derive insights and document findings.**